

Reinforcement Learning

Assignment 2 - Lunar Lander

Authors:

Martin Nilsson, *School of Engineering*, Jönköping University

Besan Ewir, *School of Engineering*, Jönköping University

Vladimir Giustacchini, *School of Engineering*, Jönköping University

Jorge Cuello, *School of Engineering*, Jönköping University

September 23, 2026

Contents

1	Theoretical Part	2
1.1	Question 1	2
1.1.1	Concept of the Q-learning update rule	2
1.1.2	Relation to the Bellman Optimality Equation	3
1.1.3	Potential challenges in using Q-learning	3
1.2	Question 2	4
1.3	Question 3	5
1.3.1	Target Network	5
1.3.2	Target Network Training Loop Example	6

1 Theoretical Part

1.1 Question 1

Q1: “Explain the concept of the Q-learning update rule. How does it relate to the Bellman optimality equation? What are the potential challenges in using this update rule? “

1.1.1 Concept of the Q-learning update rule

Q-learning is an off-policy temporal-difference (TD) control algorithm. Its main goal is to learn an action-value function Q that directly approximates q_* from sampled experience, without requiring a model of the environment's dynamics.

The update rule for tabular Q-learning is defined as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

The key components of this update rule are:

- $Q(S_t, A_t)$: the current estimated value of taking action A_t in state S_t .
- $\alpha \in (0, 1]$: the step-size parameter, or learning rate.
- R_{t+1} : the immediate scalar reward received after taking action A_t .
- $\gamma \in [0, 1)$: the discount rate for future rewards.
- $\max_a Q(S_{t+1}, a)$: the maximum estimated value across all possible actions a in the resulting next state S_{t+1} .
- $R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)$: the TD error, δ_t , representing the gap between the updated target estimate and the current estimate.

The defining feature of Q-learning is its off-policy nature: the learned action-value function Q converges toward q_* regardless of the behavior policy used to select actions during exploration.

1.1.2 Relation to the Bellman Optimality Equation

The Bellman optimality equation for the optimal action-value function q_* is:

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$$

where $p(s', r \mid s, a)$ represents the joint probability of transitioning to state s' with reward r , given current state s and action a .

The Q-learning update rule relates directly to this equation in the following ways:

- **Update rule construction:** Q-learning's update rule is obtained by turning the Bellman optimality equation for q_* into a step-by-step update rule that can be applied incrementally, rather than solved for directly.
- **Sample updates vs. expected updates:** Dynamic programming solves the Bellman optimality equation using *expected* updates, which require full knowledge of the transition dynamics $p(s', r \mid s, a)$ to average over every possible next state. Q-learning instead uses *sample* updates: it plugs in one real, observed transition $(S_t, A_t, R_{t+1}, S_{t+1})$ from the environment in place of the full expectation, letting repeated sampling substitute for exact averaging.
- **Target policy:** The Bellman optimality equation assumes the best possible action is always taken at the next state, which is exactly what $\max_{a'} q_*(s', a')$ represents. Q-learning mirrors this by using $\max_a Q(S_{t+1}, a)$ in its own update target, evaluating a greedy target policy even though the action actually taken next may come from a more exploratory behavior policy (e.g. ϵ -greedy).

1.1.3 Potential challenges in using Q-learning

While Q-learning has strong convergence guarantees in tabular settings, provided all state-action pairs are visited infinitely often, several practical challenges can arise:

- **The instability:** When Q-learning is combined with function approximation (e.g. neural networks or linear models), it brings together elements like: function approximation, bootstrapping (updating estimates using other estimates), and off-policy learning. This combination can break convergence guarantees, causing value estimates to fluctuate or diverge.

- **Suboptimal online performance during exploration:** Because Q-learning learns about the optimal (greedy) policy while actually behaving according to an exploratory policy (e.g. ϵ -greedy), its performance *during training* can be poor in environments with high-risk regions. The classic example is the Cliff Walking task, where exploratory random actions near the optimal path can cause frequent falls, compared to safer on-policy algorithms such as Sarsa.
- **Continual exploration requirement:** Convergence to q_* is only guaranteed if every state-action pair continues to be visited infinitely often, which requires careful tuning of the exploration strategy.

1.2 Question 2

Q2: “Discuss the role of experience replay in DQN. How does it help to address the correlation problem in reinforcement learning? “

Experience Replay

Experience Replay is one of the critical elements of Deep Q-Networks (DQN), where the agent saves its experiences (s, a, r, s') in the memory, which can be used during learning rather than learning from the latest experience only. In the training process, samples are randomly selected from the experience replay buffer.

The main benefit of using experience replay is that it helps reduce the correlation problem, since in reinforcement learning, the experiences that come one after another are highly correlated. Random sampling mixes experiences from different time steps, which reduces the correlation between successive updates and makes learning more stable.

Experience Replay also allows the same experience to be used multiple times while the agent still learning, making learning more efficient and reducing the variance of the updates.

1.3 Question 3

Q3: “Explain the concept of the target network in DQN. How does it help to stabilize the learning process? “

1.3.1 Target Network

The target network is a copy of the Q-network that is used to calculate the target Q-values during training. The target network is updated less frequently than the Q-network, which helps to stabilize the learning process by reducing the correlation between the Q-values and the target values.

Keep in mind that the *target network* is **NOT** necessarily a different architecture or a separate trained network. In most cases it is simply a periodically updated copy of the online network.

In a neural network with θ parameters, we have $Q(s, a; \theta)$ and the target becomes

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

But since we use the same network to calculate the target, it can lead to instability and divergence in the learning process because the target would move with the parameters. To address this issue, we introduce a *target network* with parameters θ^- , which is a copy of the Q-network that is updated less frequently (e.g., every C steps). The target becomes

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

- $Q(s, a; \theta)$ (**Online network** → the network that is being trained)
- $Q(s', a'; \theta^-)$ (**Target network** → the network that is used to calculate the target Q-values)

1.3.2 Target Network Training Loop Example

A simplified DQN training loop with a target network can be described as follows:

DQN Target-Network Pseudocode

Initialize online network θ

Initialize target network $\theta^- = \theta$

Repeat:

1. Take an action using the online network.
2. Store transition (s, a, r, s') in the Experience/Replay buffer.
3. Sample a mini-batch from the replay buffer.
4. Calculate targets using the target network:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-).$$

5. Update the online network θ to minimize:

$$\left(y - Q(s, a; \theta)\right)^2.$$

6. Every C steps, update the target network:

$$\theta^- \leftarrow \theta.$$

The main takeaway is that the target network helps to stabilize the learning process by providing a more stable target for the Q-value updates, which reduces the risk of divergence and improves convergence.